

Praktikum 2 Sistem Operasi: Proses

Informasi Praktikum

- **Mata Kuliah:** Sistem Operasi
 - **Sistem Operasi:** Linux Ubuntu
 - **Bahasa Pemrograman:** C
 - **Dosen Pengampu:** Triawan Adi Cahyanto, M.Kom
 - **Asisten Dosen:** Atidhira Habibillah dan Taqiyyuddin
-

Topik 1: Process Concept dan Process Operations

Tujuan Pembelajaran

1. Memahami konsep proses dan siklus hidupnya
2. Mampu membuat dan mengelola proses menggunakan system call
3. Memahami hubungan parent-child process
4. Menganalisis Process ID (PID) dan Process Control Block

Teori Singkat

Proses adalah program yang sedang dieksekusi. Setiap proses memiliki PID unik dan dapat membuat proses baru (child process) menggunakan system call `fork()`.

Persiapan

```
# Install compiler jika belum ada
sudo apt update
sudo apt install build-essential
```

Percobaan 1.1: Membuat Proses Sederhana

Langkah Kerja:

1. Buat file `process_basic.c`:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;

    printf("Program dimulai\n");
    printf("PID proses ini: %d\n", getpid());
```

```
    printf("PID parent: %d\n", getppid());

    return 0;
}
```

2. Compile dan jalankan:

```
gcc process_basic.c -o process_basic
./process_basic
```

Percobaan 1.2: Fork - Parent dan Child Process

Langkah Kerja:

1. Buat file `fork_demo.c`:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;

    printf("Sebelum fork()\n");
    printf("PID: %d\n\n", getpid());

    pid = fork();

    if (pid < 0) {
        fprintf(stderr, "Fork gagal!\n");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("CHILD PROCESS\n");
        printf("PID saya: %d\n", getpid());
        printf("PID parent saya: %d\n", getppid());
    }
    else {
        // Parent process
        printf("PARENT PROCESS\n");
        printf("PID saya: %d\n", getpid());
        printf("PID child saya: %d\n", pid);
    }

    printf("Proses %d selesai\n\n", getpid());

    return 0;
}
```

2. Compile dan jalankan:

```
gcc fork_demo.c -o fork_demo
./fork_demo
```

Percobaan 1.3: Process Termination dengan wait()

Langkah Kerja:

1. Buat file `process_wait.c`:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    pid = fork();

    if (pid == 0) {
        // Child process
        printf("Child: PID saya %d, akan tidur 3 detik\n", getpid());
        sleep(3);
        printf("Child: Selesai!\n");
        return 42; // Exit code
    }
    else {
        // Parent process
        printf("Parent: Menunggu child (PID: %d) selesai...\n", pid);
        wait(&status);

        if (WIFEXITED(status)) {
            printf("Parent: Child selesai dengan exit code: %d\n",
                WEXITSTATUS(status));
        }
    }

    return 0;
}
```

Percobaan 1.4: Multiple Child Processes

Langkah Kerja:

1. Buat file `multiple_fork.c`:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int i;
    pid_t pid;
    int num_children = 3;

    printf("Parent PID: %d\n\n", getpid());

    for (i = 0; i < num_children; i++) {
        pid = fork();

        if (pid == 0) {
            // Child process
            printf("Child %d: PID = %d, Parent PID = %d\n",
                i+1, getpid(), getppid());
            sleep(i + 1); // Simulasi pekerjaan
            printf("Child %d selesai\n", i+1);
            return i;
        }
    }

    // Parent menunggu semua child
    for (i = 0; i < num_children; i++) {
        int status;
        pid_t child_pid = wait(&status);
        printf("Parent: Child dengan PID %d selesai\n", child_pid);
    }

    printf("\nParent: Semua child selesai\n");

    return 0;
}
```

Topik 2: Interprocess Communication (IPC)

Tujuan Pembelajaran

1. Memahami konsep komunikasi antar proses
2. Mengimplementasikan Shared Memory IPC
3. Mengimplementasikan Message Passing dengan Pipes
4. Membandingkan performa kedua metode

Percobaan 2.1: Pipes - Communication One Way

Langkah Kerja:

1. Buat file `pipe_demo.c`:

```

#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    int pipefd[2]; // pipefd[0] = read, pipefd[1] = write
    pid_t pid;
    char write_msg[] = "Hello dari Parent!";
    char read_msg[100];

    // Buat pipe
    if (pipe(pipefd) == -1) {
        perror("Pipe gagal");
        return 1;
    }

    pid = fork();

    if (pid == 0) {
        // Child process - membaca dari pipe
        close(pipefd[1]); // Tutup write end

        read(pipefd[0], read_msg, sizeof(read_msg));
        printf("Child menerima: %s\n", read_msg);

        close(pipefd[0]);
    }
    else {
        // Parent process - menulis ke pipe
        close(pipefd[0]); // Tutup read end

        printf("Parent mengirim: %s\n", write_msg);
        write(pipefd[1], write_msg, strlen(write_msg) + 1);

        close(pipefd[1]);
        wait(NULL);
    }

    return 0;
}

```

Percobaan 2.2: Named Pipes (FIFO) - Two Way Communication

Langkah Kerja:

1. Buat file `fifo_writer.c`:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```

#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

#define FIFO_FILE "/tmp/my_fifo"

int main() {
    int fd;
    char buffer[100];

    // Buat FIFO jika belum ada
    mkfifo(FIFO_FILE, 0666);

    printf("Writer: Menunggu reader...\n");
    fd = open(FIFO_FILE, O_WRONLY);

    printf("Masukkan pesan: ");
    fgets(buffer, sizeof(buffer), stdin);

    write(fd, buffer, strlen(buffer) + 1);
    printf("Writer: Pesan terkirim\n");

    close(fd);

    return 0;
}

```

2. Buat file `fifo_reader.c`:

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

#define FIFO_FILE "/tmp/my_fifo"

int main() {
    int fd;
    char buffer[100];

    printf("Reader: Membuka FIFO...\n");
    fd = open(FIFO_FILE, O_RDONLY);

    read(fd, buffer, sizeof(buffer));
    printf("Reader: Menerima pesan: %s\n", buffer);

    close(fd);
    unlink(FIFO_FILE); // Hapus FIFO

    return 0;
}

```

3. Compile dan jalankan di 2 terminal:

```
# Terminal 1
gcc fifo_reader.c -o reader
./reader

# Terminal 2
gcc fifo_writer.c -o writer
./writer
```

Percobaan 2.3: Shared Memory IPC

Langkah Kerja:

1. Buat file `shm_writer.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_SIZE 1024

int main() {
    key_t key;
    int shmid;
    char *shared_memory;
    char message[100];

    // Generate unique key
    key = ftok("shmfile", 65);

    // Create shared memory
    shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);

    // Attach to shared memory
    shared_memory = (char*) shmat(shmid, NULL, 0);

    printf("Writer: Masukkan pesan: ");
    fgets(message, sizeof(message), stdin);

    // Write to shared memory
    strcpy(shared_memory, message);
    printf("Writer: Data ditulis ke shared memory\n");

    // Detach from shared memory
    shmdt(shared_memory);
}
```

```
    return 0;
}
```

2. Buat file `shm_reader.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_SIZE 1024

int main() {
    key_t key;
    int shmid;
    char *shared_memory;

    // Generate same key
    key = ftok("shmfile", 65);

    // Access shared memory
    shmid = shmget(key, SHM_SIZE, 0666);

    // Attach to shared memory
    shared_memory = (char*) shmat(shmid, NULL, 0);

    // Read from shared memory
    printf("Reader: Data dari shared memory: %s\n", shared_memory);

    // Detach and remove shared memory
    shmdt(shared_memory);
    shmctl(shmid, IPC_RMID, NULL);

    return 0;
}
```

3. Compile dan jalankan:

```
# Buat file untuk ftok
touch shmfile

# Compile
gcc shm_writer.c -o shm_writer
gcc shm_reader.c -o shm_reader

# Terminal 1
./shm_writer
```



```
# Terminal 2
./shm_reader
```

Percobaan 2.4: Perbandingan Performa IPC

Langkah Kerja:

1. Buat file `ipc_benchmark.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/time.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define DATA_SIZE 1000000
#define SHM_SIZE DATA_SIZE * sizeof(int)

double get_time() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return tv.tv_sec + tv.tv_usec / 1000000.0;
}

void test_pipe() {
    int pipefd[2];
    pid_t pid;
    int data[DATA_SIZE];
    double start, end;

    pipe(pipefd);
    start = get_time();

    pid = fork();

    if (pid == 0) {
        close(pipefd[1]);
        read(pipefd[0], data, sizeof(data));
        close(pipefd[0]);
        exit(0);
    }
    else {
        close(pipefd[0]);

        for (int i = 0; i < DATA_SIZE; i++) {
            data[i] = i;
        }

        write(pipefd[1], data, sizeof(data));
    }
}
```

```
        close(pipefd[1]);
        wait(NULL);

        end = get_time();
        printf("Pipe: %.6f detik\n", end - start);
    }
}

void test_shared_memory() {
    key_t key = ftok("shmfile", 65);
    int shmid;
    int *shared_data;
    pid_t pid;
    double start, end;

    shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);
    start = get_time();

    pid = fork();

    if (pid == 0) {
        shared_data = (int*) shmat(shmid, NULL, 0);
        // Child hanya membaca
        int sum = 0;
        for (int i = 0; i < 100; i++) {
            sum += shared_data[i];
        }
        shmdt(shared_data);
        exit(0);
    }
    else {
        shared_data = (int*) shmat(shmid, NULL, 0);

        for (int i = 0; i < DATA_SIZE; i++) {
            shared_data[i] = i;
        }

        wait(NULL);
        end = get_time();

        printf("Shared Memory: %.6f detik\n", end - start);

        shmdt(shared_data);
        shmctl(shmid, IPC_RMID, NULL);
    }
}

int main() {
    printf("Benchmarking IPC Methods dengan %d integers\n\n", DATA_SIZE);

    test_pipe();
    test_shared_memory();
}
```

```
    return 0;  
}
```

TUGAS AKHIR PRAKTIKUM

Tugas 1: Sistem Manajemen Proses

Buat program C yang mensimulasikan sistem manajemen proses sederhana dengan fitur:

- Membuat multiple child processes
- Setiap process melakukan tugas berbeda (sorting, searching, calculation)
- Parent process mengkoordinasi dan mengumpulkan hasil
- Tampilkan PID, execution time, dan status setiap proses

Tugas 2: Chat Application dengan IPC

Implementasikan aplikasi chat sederhana menggunakan:

- Shared Memory atau Message Queue untuk menyimpan pesan
- Multiple processes sebagai pengguna
- Mutex/Semaphore untuk sinkronisasi
- Fitur: kirim pesan, lihat history, exit

Contoh Output:

```
=== CHAT ROOM ===  
User1> Halo semua!  
User2> Hai User1!  
User1> Bagaimana kabarmu?  
User2> Baik, terima kasih
```

LAPORAN PRAKTIKUM

Format Laporan

1. Cover

- Judul Praktikum
- Nama dan NIM
- Program Studi
- Tanggal

2. Tujuan

- Tujuan dari setiap percobaan

3. Dasar Teori

- Konsep proses
- IPC methods

4. Percobaan Untuk setiap percobaan:

- Source code lengkap
- Screenshot output program
- Analisis hasil
- Jawaban tugas

5. Tugas Akhir

- Source code lengkap
- Penjelasan algoritma (kalau ada)
- Screenshot output
- Analisis performa

6. Kesimpulan

- Pembelajaran dari praktikum
- Perbandingan metode IPC

TIPS & TROUBLESHOOTING

Compile Errors

Error: undefined reference to pthread_create

```
# Solusi: Tambahkan flag -pthread  
gcc program.c -o program -pthread
```

Error: permission denied

```
# Solusi: Buat file executable  
chmod +x program
```

Runtime Issues

Segmentation Fault di Shared Memory

- Pastikan key file untuk ftok() sudah dibuat
- Cek ukuran shared memory mencukupi
- Gunakan proper error checking

Program Hang (Deadlock)

- Gunakan timeout untuk testing: `timeout 10 ./program`

- Tekan Ctrl+C untuk terminate
- Cek urutan lock/unlock mutex

Race Condition Tidak Terdeteksi

- Tingkatkan NUM_INCREMENTS
- Jalankan multiple kali
- Gunakan tool seperti Valgrind

Debugging Commands

```
# Melihat proses yang berjalan
ps aux | grep program_name

# Melihat shared memory segments
ipcs -m

# Menghapus shared memory
ipcrm -m <shmid>

# Melihat thread dari proses
ps -T -p <pid>

# Monitor resource usage
top -H
htop
```

Useful Tools

```
# Install debugging tools
sudo apt install valgrind gdb

# Check memory leaks
valgrind --leak-check=full ./program

# Debugging dengan gdb
gdb ./program
(gdb) run
(gdb) backtrace
```

REFERENSI

System Calls Documentation

```
# Baca manual pages
man fork
```

```
man pthread_create  
man shmget  
man pipe  
man semaphore
```

Online Resources

- Linux Man Pages: <https://man7.org/linux/man-pages/>
- POSIX Threads: <https://pubs.opengroup.org/onlinepubs/>
- GNU C Library: <https://www.gnu.org/software/libc/manual/>

Books

- "Operating System Concepts" - Silberschatz
 - "Advanced Programming in the UNIX Environment" - Stevens
 - "The Linux Programming Interface" - Kerrisk
-